

Lecture: Ternary Search Algorithm — From Zero to Advanced

1. Introduction

Ternary Search is an advanced searching algorithm based on the **Divide and Conquer** strategy.

Instead of dividing the search interval into **two halves** (like Binary Search), it divides it into **three parts**, reducing the range faster in certain cases.

Definition:

Ternary Search repeatedly splits the sorted array into three parts using two mid-points and then decides which part contains the target.

Real-life analogy:

Imagine looking for a page in a sorted book by dividing it into **three equal sections**, opening at one-third and two-thirds, and deciding which portion contains the page you want.

2. Key Characteristics

- **Type:** Divide-and-Conquer Search
- **Input Requirement:** Data must be **sorted** (ascending or descending).
- **Direction:** Jumps across three sections instead of two.
- **Time Complexity:** $O(\log_3 n)$ (similar to Binary Search, just base 3).
- **Space Complexity:**
 - Iterative: $O(1)$.
 - Recursive: $O(\log_3 n)$.

3. Step-by-Step Working

1. Start with $low = 0$ and $high = n-1$.
2. Calculate two midpoints:
 - $mid1 = low + (high - low) // 3$

- $\text{mid2} = \text{high} - (\text{high} - \text{low}) // 3$
- 3. Compare $\text{arr}[\text{mid1}]$ and $\text{arr}[\text{mid2}]$ with the target:
 - If $\text{arr}[\text{mid1}] == \text{target} \rightarrow \text{return mid1}$.
 - If $\text{arr}[\text{mid2}] == \text{target} \rightarrow \text{return mid2}$.
- 4. If $\text{target} < \text{arr}[\text{mid1}] \rightarrow \text{search in the first part } [\text{low} \dots \text{mid1}-1]$.
- 5. Else if $\text{target} > \text{arr}[\text{mid2}] \rightarrow \text{search in the third part } [\text{mid2}+1 \dots \text{high}]$.
- 6. Else $\rightarrow \text{search in the middle part } [\text{mid1}+1 \dots \text{mid2}-1]$.
- 7. Repeat until $\text{low} > \text{high} \rightarrow \text{target not found}$.

4. Pseudocode

TernarySearch(arr, low, high, target):

while low <= high:

$\text{mid1} = \text{low} + (\text{high} - \text{low}) // 3$

$\text{mid2} = \text{high} - (\text{high} - \text{low}) // 3$

if $\text{arr}[\text{mid1}] == \text{target}$:

 return mid1

if $\text{arr}[\text{mid2}] == \text{target}$:

 return mid2

if $\text{target} < \text{arr}[\text{mid1}]$:

$\text{high} = \text{mid1} - 1$

elif $\text{target} > \text{arr}[\text{mid2}]$:

$\text{low} = \text{mid2} + 1$

else:

$\text{low} = \text{mid1} + 1$

$\text{high} = \text{mid2} - 1$

return -1

5. Complexity Analysis

- **Best Case:** $O(1)$ → target is at mid1 or mid2.
- **Worst Case:** $O(\log_3 n)$.
- **Average Case:** $O(\log_3 n)$.
- **Space Complexity:**
 - Iterative: $O(1)$.
 - Recursive: $O(\log_3 n)$.



Note:

Although the complexity looks better (log base 3), in practice **Binary Search is faster** due to fewer comparisons per step.

6. Handling Edge Cases

1. Empty list → return -1.
2. Single element list → check directly.
3. Multiple occurrences → modify algorithm to find first/last.
4. Works with numbers, strings, or objects (if sorted).
5. Incorrect input (unsorted list) → invalid results.

7. Variations of Ternary Search

- **Recursive Ternary Search** — using function calls.
- **First & Last Occurrence Search** in sorted arrays.
- **Ternary Search on Answer Space** — used in optimization problems like finding maxima/minima of unimodal functions.

8. When to Use Ternary Search

Best suited for:

- Unimodal functions (where function decreases, then increases, or vice versa).
- Optimization problems (finding min/max).
- Sorted arrays (though Binary Search is usually preferred).

Avoid if:

- Large datasets requiring frequent searches (Binary Search is generally more efficient).
- Small arrays where Linear Search may be simpler.

9. Advanced Usage

- **Ternary Search on Continuous Space** → Used in mathematical optimization.
- **Finding peak elements** in unimodal arrays.
- Used in **computational geometry** and **numerical methods**.

10. Real-World Applications

- Optimizing functions in physics & engineering.
- Searching in unimodal datasets.
- Machine learning — hyperparameter optimization.
- Game development — minimizing/maximizing cost functions.

11. Summary Table

Feature	Ternary Search
Works on sorted?	Yes
Works on unsorted?	No
Time Complexity	$O(\log_3 n)$
Space Complexity	$O(1)$ iterative, $O(\log_3 n)$ recursive
Best Case	$O(1)$
Worst Case	$O(\log_3 n)$

12. Practice Tasks

1. Implement recursive ternary search.
2. Modify algorithm to return all occurrences of target.
3. Write ternary search for a unimodal function (find max/min).
4. Compare execution time of Binary vs Ternary Search on large datasets.

5. Implement ternary search in a rotated sorted array.

13. Closing Thought

Ternary Search is a fascinating algorithm.

While Binary Search is usually more practical, Ternary Search shines in **optimization problems** and advanced computational tasks.

Mastering it helps in understanding **search space reduction** and the **divide-and-conquer paradigm** at a deeper level.